

Gestão Financeira de Construção Civil

Davide Moreno da Silva a24717

<Nome do Aluno e Número Mecnográfico - Estilo NomeAluno>

Trabalho realizado sob a orientação de

<Professor Nome do Orientador - Estilo NomeOrientador>

<Professor Nome do Co-Orientador - Estilo NomeOrientador >

Licenciatura em Tecnologias Digitais e Gestão

2025/2026

Gestão Financeira de Construção Civil

Relatório da UC de Projecto de Informática
Licenciatura em Tecnologias Digitais e Gestão
Escola Superior de Tecnologia e Gestão

Davide Moreno da Silva

Junho de 2026

A Escola Superior de Tecnologia e Gestão não se responsabiliza pelas opiniões expressas neste relatório.

Certifico que li este relatório e que na minha opinião, é adequado no seu conteúdo e forma como demonstrador do trabalho desenvolvido no âmbito da UC de Projecto de Informática.

Erro! Autorreferência de marcador inválida. Orientador

Certifico que li este relatório e que na minha opinião, é adequado no seu conteúdo e forma como demonstrador do trabalho desenvolvido no âmbito da UC de Projecto de Informática.

Co-Orientador

Certifico que li este relatório e que na minha opinião, é adequado no seu conteúdo e forma como demonstrador do trabalho desenvolvido no âmbito da UC de Projecto de Informática.

Arguente

Aceite para avaliação da UC de Projecto de Informática

Dedicatória

Dedico este Trabalho à minha Família e a todos os que de uma maneira ou outra apoiaram esta jornada académica

Agradecimentos

Gostaria de expressar o meu sincero agradecimento aos meus orientadores por toda a orientação, disponibilidade e partilha de conhecimento que enriqueceram o presente trabalho durante todo o processo. Estendo este agradecimento também aos meus colegas de curso pelo companheirismo e entreaajuda, bem como a todos os docentes, não docentes e à instituição pelo ambiente propício ao desenvolvimento deste projeto como conclusão do meu percurso académico.

Por fim, deixo uma agradecimento muito especial à minha família e amigos, cujo o apoio incondicional, compreensão e incentivo constante foram fundamentais para superar os desafios e concluir com sucesso esta importante etapa académica. A todos os que acreditaram na minhas minha capacidades, o meu maior obrigado. Aos que, pelo contrário, duvidaram do meu percurso, agradeço igualmente, pois o vosso ceticismo serviu de combustível para cruzar a meta de uma corrida que muitos julgavam perdida desde o início.

Resumo

A gestão financeira eficaz é um dos desafios permanentes para as empresas de construção civil, dadas a elevada complexidade das obras e o enorme número de variáveis operacionais envolvidas. O projeto consistiu no desenvolvimento de uma plataforma tecnológica orientada para a gestão integrada e eficaz dos aspetos financeiros desse setor, no sentido de garantir a sustentabilidade do negócio através de uma visão mais clara sobre os fluxos de caixa, orçamentos e custos. A metodologia de trabalho utilizada foi estruturada em seis etapas principais: estudo preliminar, identificação de requisitos, análise detalhada, projeto, implementação e teste, culminando na produção técnica e guias de utilização. Como principal desenvolvimento, a solução implementada proporcionou o controlo orçamental com alocação eficiente de custos, a gestão automatizada do fluxo de caixa (entrada e saída), o controlo dos pagamentos a fornecedores e subcontratos, bem como a geração de relatórios em tempo real e a geração de previsões financeiras baseada nos dados históricos. Constatou-se que a plataforma possibilita uma otimização significativa dos processos analíticos e uma redução dos riscos operacionais, dotando as empresas de uma ferramenta intuitiva que permite a tomada de decisões estratégicas em tempo útil.

Palavras-chave: Gestão Financeira, Construção Civil, Controlo Orçamental, Fluxo de Caixa, Tomada de Decisão

Abstract

Effective financial management is one of the ongoing challenges faced by construction companies, given the high complexity of projects and the large number of operational variables involved. The project consisted in the development of a technology-driven platform designed to ensure integrated and efficient management of the financial aspects of this sector, with the goal of guaranteeing business sustainability through clearer visibility over cash flows, budgets, and costs. The working methodology was structured into six main stages: preliminary study, requirements identification, detailed analysis, design, implementation, and testing, culminating in the production of technical documentation and user guides. As its main outcome, the implemented solution enabled budget control with efficient cost allocation, automated cash-flow management (inflows and outflows), monitoring of payments to suppliers and subcontractors, as well as real-time reporting and financial forecasting based on historical data. It was concluded that the platform provides significant optimisation of analytical processes and a reduction of operational risks, equipping companies with an intuitive tool that supports timely strategic decision-making.

Keywords: Financial Management, Construction Industry, Budget Control, Cash Flow, Decision-Making

Conteúdo

1	Introdução	Erro! Marcador não definido.
1.1	Contextualização	Erro! Marcador não definido.
1.2	Motivação e Problema.....	Erro! Marcador não definido.
1.3	Objetivos	Erro! Marcador não definido.
1.4	Estrutura do Relatório	2
2	Estado da Arte	Erro! Marcador não definido.
2.1	Soluções Existentes	Erro! Marcador não definido.
2.1	Tecnologias Relevantes	Erro! Marcador não definido.
3	Análise e Design do Sistema.....	Erro! Marcador não definido.
3.1	Requisitos	Erro! Marcador não definido.
3.2	Arquitetura do Sistema	5
3.3	Modelo de Dados	6
4	Implementação.....	7Erro! Marcador não definido.
4.1	Backend – Django REST Framework.....	7
4.2	Frontend - React	8
4.3	Infraestrutura - Docker.....	9 Erro! Marcador não definido.
5	Resultados e Discussão	10Erro! Marcador não definido.
5.1	Funcionalidades Implementadas	10Erro! Marcador não definido.
5.2	Teste.....	11
6	Conclusão	12

<u>6.1 Considerações Finais</u>	12
6.2 Trabalho Futuro	12
6.3 Infraestrutura - Docker	12
Referências Bibliográficas	Erro! Marcador não definido. <u>3</u>
Anexo A – API Endpoint	A- Erro! Marcador não definido.

Lista de Tabelas

Tabela 3.1 --- Requisitos Funcionais da ConstruManage.

Tabela 3.2 --- Stack Tecnológico.

Tabela 4.1 --- Modelos de Dados e Atributos Principais.

Tabela 5.1 --- Funcionalidades Implementadas

Lista de Figuras

Figura 3 .1 --- Diagrama de Arquitetura da Plataforma

Figura 3 .2 --- Diagrama Entidade - Relacionamento

Figura 3 .1 --- Fluxo de Autenticação JWT

Lista de Abreviações

API Application Programming Interface

CRUD Create, Read, Update, Delete

DRF Django REST Framework

ER Entidade – Relacionamento

IPB Instituto Politécnico de Bragança

JWT JSON Web Token

ORM Object – Relational Mapping

REST Representation State Transfer

SPA Single Page Application

Capítulo 1

1 Introdução

1.1 Contextualização

A gestão financeira das empresas de construção civil representam um dos desafios mais complexos do setor, dado o elevado número de obras simultâneas, a multiplicidade de fornecedores, a variabilidade dos custos e a necessidade de previsão rigorosa dos fluxos de caixa. As ferramentas tradicionais – baseadas em folhas de cálculo ou em software genérico de gestão – revelam-se frequentemente inadequadas para responder à complexidade e às especificidades do setor.

Neste contexto, o projeto Construmanage surge como resposta a uma necessidade real e concreta, propondo o desenvolvimento de uma plataforma web dedicada à gestão financeira integrada de empresas de construção civil, com funcionalidade adaptada à complexidade e às especificidades do setor.

1.2 Motivação e Problema

A motivação principal para o desenvolvimento desta plataforma reside na constatação de que a maioria das pequenas e médias empresas de construção civil portuguesa não dispõe de ferramentas adequadas para monitorizar em tempo real os seus indicadores financeiros por obra. Esta lacuna conduz frequentemente a desvios orçamentais não detetados

atempadamente, atrasos em pagamentos a fornecedores e dificuldades na elaboração de previsões financeiras fiáveis.

O problema central pode, assim, ser formulado da seguinte forma: como proporcionar às empresas de construção civil uma solução tecnológica acessível, integrada e intuitiva para a gestão financeira das suas obras, desde o controlo orçamentação ao fluxo de caixa e às previsões mensais.

1.3 Objetivos

Os objetivos gerais do projeto são:

- Desenvolver uma plataforma web completa para gestão financeira de empresas de construção civil.
- Implementar módulos de dashboard em tempo real, controlo orçamental, fluxo de caixa, gestão de fornecedores e previsões financeiras.
- Garantir autenticação segura com tokens JWT.
- Disponibilizar a plataforma em ambiente Docker para facilidade de instalação e portabilidade

Como objetivos específicos destacam-se:

- Implementar uma API REST com Django REST Framework, expondo endpoints de autenticação e CRUD para os principais recursos.
- Desenvolver uma SPA em React 18 com visualizações baseadas em Recharts.
- Configurar orquestração de serviços com Docker Compose(frontend, backend, PostgreSQL).
- Implementar refresh automático de tokens JWT via interceptor Axios.

1.4 Estrutura do Relatório

Este relatório está organizado em seis capítulos principais. O Capítulo 2 apresenta o estado da arte, analisando soluções existentes e tecnologias relevantes. O Capítulo 3 descreve a análise e o design do sistema, incluindo requisitos, arquitetura e modelo de dados. O Capítulo 4 detalha a implementação dos módulos backend, frontend e de infraestrutura. O Capítulo 5 apresenta os resultados obtidos e discute as funcionalidades implementadas. O Capítulo 6 encerra o relatório com as conclusões e perspectivas de trabalho futuro.

2 Estado da Arte

2.1 Soluções Existentes

A oferta de software de gestão financeira para a construção civil é dominada por soluções de grande dimensão, como o SAP ERP, o Primavera BSS e o PHC CS, que, embora abrangentes, apresentam elevados custos de licenciamento e complexidade de implementação, tornando-as inacessíveis para a maioria das pequenas e médias empresas do setor.

Existem igualmente soluções específicas para a construção, como o Buildtrend, o Procore e o CoConstruct, orientadas principalmente para o mercado anglo-saxónico e com funcionalidades centradas na gestão de projeto, mas com limitações ao nível da integração financeira adaptada ao contexto português.

O recurso a folhas de cálculo Microsoft Excel continua a ser a abordagem mais utilizada pelas PME nacionais, com as limitações inerentes de falta de centralização de dados, ausência de controlo de acesso, dificuldade de colaboração em tempo real e riscos de erros manuais.

A ConstrManage posiciona-se como uma alternativa acessível, open-source e adaptada ao contexto nacional, combinando as funcionalidades essenciais de gestão financeira com uma interface moderna e de fácil utilização.

2.2 Tecnologias Relevantes

A seleção de stack tecnológico baseou-se em critérios de maturidade, suporte da comunidade, adequação ao problema e experiência prévia da equipa de desenvolvimento.

Camada	Tecnologia	Versão	Justificação
Backend	Python + Django	3.12/5.x	Maturidade, Orm robusto, ecossistema DRF
API REST	Django REST Framework	3.15	Serialização, ViewSets, autenticação JWT
Frontend	React + Vite	18 / 5	Componentização SPA, hot-reload
Gráficos	Recharts	2.x	Integração React, gráficos responsivos
Base de Dados	PostgreSQL	16	ACID, escalabilidade, suporte Django
Infraestrutura	Docker + Compose	latest	Portabilidade, orquestração multi-serviço
Auth	Simple JWT	5 . X	Tokens JWT, refresh automático

Tabela 3.2 - Stack Tecnológico

3 Análise e Design do Sistema

3.1 Requisitos

3.1.1 Requisitos Funcionais

A análise de requisitos foi conduzida com base na identificação das necessidades dos utilizadores-alvo (gestores financeiros e administradores de empresas de construção civil). A Tabela 3.1 sintetiza os principais requisitos funcionais identificados.

ID	Requisito	Prioridade
RF01	Autenticação e registo de utilizadores com JWT	Alta
RF02	Dashboard com indicadores financeiros em tempo real	Alta
RF03	Gestão de obras(CRUD) com controlo orçamental	Alta
RF04	Registo e categorização de transações financeiras	Alta
RF05	Gestão de fornecedores e pagamentos com alertas e atrasos	Média
RF06	Previsões financeiras mensais a 6 meses	Média
RF07	Visualização gráfica de receitas, despesas e margens	Alta
RF08	Suporte multi-utilizador com isolamento de dados por sessão	Baixa

Tabela 3.1 - Requisitos Funcionais de ConstruManage.

3.1.2 Requisitos não Funcionais

- Disponibilidade: a plataforma deve estar acessível via browser sem instalação de software adicional.
- Segurança: autenticação obrigatória em todos os endpoints protegidos, tokens com expiração configurável.
- Portabilidade: toda a aplicação deve ser executável via Docker Compose num único comando.
- Manutenibilidade: arquitetura modular, código comentado, separação clara entre camadas.
- Usabilidade: interface responsiva, navegação intuitiva, feedback visual imediato ao utilizador.

3.2 Arquitetura do Sistema

A plataforma segue uma arquitetura cliente-servidor desacoplada, orquestrada pelo Docker Compose com três serviços independentes: frontend (React + Nginx), backend (Django + Gunicorn) e base de dados (PostgreSQL 16). O Nginx atua simultaneamente como servidor de SPA React e como reverse proxy para a API Django.

Tabela 3.2 - Requisitos Funcionais de ConstruManage.

[Frontend React + Nginx :80] →API→ [Backend Django + Gunicorn :8000] →SQL→ [PostgreSQL :5432]

O fluxo de comunicação segue o padrão: o utilizador acede ao frontend via browser (porta 80); o Nginx serve os assets estáticos de SPA e encaminha os pedidos com prefixo /api/ para o backend(porta 8000): o backend processa os pedidos, interage com a base de dados PostgreSQL e devolve resposta JSON.

3.3 Modelo de Dados

O modelo de dados da ConstruManage é composto por quatro entidades principais, implementadas como modelos Django e persistidas em PostgreSQL.

3.3.1 Entidade e Relações

Entidades	Atributos Principais	Relações
Obra	nome, orcamento_aprovado, custo_atual, progresso, status, data_inicio, data_fim_prevista	1:N com Transação. Fornecedor
Transação	descricao, tipo (E/S), valor, categoria, data	N:1 com Obra
Fornecedor	nome, servico, prazo_pagamento, valor, status_pagamento	N:1 com Obra
Previsão Financeiras	mes, recebimentos_previstos, pagamentos_previstos	Independente

Tabela 4.1 - Modelos de Dados e Atributos Principais

O modelo Obra é o agregado central, ao qual se associam Transações e Fornecedores numa relação um-par-muitos. A previsão Financeira é uma entidade independente que armazena projeções mensais de recebimento e pagamentos. O modelo Obra inclui propriedades calculadas (desvio orçamental e percentagem de orçamento consumido) implementadas como propriedade Python.

4 Implementação

4.1 Backend – Django REST Framework

4.1.1 Requisitos Funcionais

O backend foi estruturado seguindo as convenções do Django, com um projeto principal (construmanager) e uma aplicação (api) responsável por todos os modelos, serializer, views e rota da API REST. A configuração é externalizada para variáveis de ambiente, lidas em runtime via os.environ.get

O ficheiro requirements.txt inclui as seguintes dependências principais:

```
django>=5.0,<6.0
djangorestframework>=3.14,<4.0
djangorestframework-simplejwt>=5.3,<6.0
django-cors-headers>=4.3,<5.0
psycopg2-binary>=2.9,<3.0
gunicorn>=21.2,<23.0
```

4.1.2 Modelos de Dados

Os quatro modelos Django (Obra, Transação, Fornecedor, PrevisãoFinanceira) foram implementados com o campo tipados, choices para enumerações, relações ForeignKey e propriedades calculadas. A seguir apresenta-se um excerto do modelo Obra.

```

class Obra(models.Model):
    STATUS_CHOICES = [
        ('em_curso', 'Em Curso'),
        ('concluida', 'Concluida'),
        ('suspensa', 'Suspensa'),
        ('planeada', 'Planeada'),
    ]

    nome = models.CharField(max_length=255)
    descricao = models.TextField(blank=True, default='')
    orcamento_aprovado = models.DecimalField(max_digits=14, decimal_places=2)
    custo_atual = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    progresso = models.IntegerField(default=0, help_text='Percentagem de progresso (0-100)')
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='planeada')
    data_inicio = models.DateField()
    data_fim_prevista = models.DateField(null=True, blank=True)
    criado_em = models.DateTimeField(auto_now_add=True)
    atualizado_em = models.DateTimeField(auto_now=True)

```

4.1.3 API REST e Autenticação JWT

A API REST expõe endpoints de autenticação (login, refresh, registo) e endpoints CRUD para cada recurso principal. A autenticação é implementada com Simple JWT utilizando tokens de acesso com duração de 1 hora e token de refresh com duração de 7 dias, ambos configuráveis via variáveis de ambiente.

O endpoint de dashboard (/api/dashboard/) agraga dados de múltiplos modelos, calculando receitas e despesas mensais, saldo atual, evolução dos últimos 6 meses, distribuição de custos por categoria e total de pagamentos vencidos.

Figura 4.1 – Fluco de Autenticação JWT

[Utilizador] → POST /api/auth/login/ → {access, refresh} → Requests com Bearer token → Interceptor Axios renova token → Expiração total → Redirect /login

4.2 Frontend – React

4.2.1 Estrutura de Componentes

O frontend foi essencialmente desenvolvido com React 18 e Vite 5, seguindo uma organização em páginas e componentes reutilizáveis. A navegação é gerida através do React Router 6, com rotas protegidas que redirecionam para /login quando o utilizador não se encontra autenticado.

Os principais componentes são:

- Dashboard.jsx – Gráficos de barras (receitas vs despesas do mês) e gráficos de pizza (distribuição de custos por categoria), com cards de indicadores principais.
- Orcamento.jsx – Tabela de obras com barras para registo de transações e tabelas com histórico.
- FluxoCaixa.jsx- Formulário interativo para registos de transações e tabela com histórico.
- Fornecedores.jsx – Lista de fornecedores com indicadores de estado de pagamento e alertas de atraso.
- Previsões.jsx – Gráfico de área com projeções a 6 meses a análise de risco de tesouraria.

4.2.2 Gestão de Estado e API

A comunicação com a API REST é centralizada no módulo api/api.js, que configura uma instância Axios com a URL base da API e intercetores para injeção automática do token JWT nos headers e para refresh automático quando o token de acesso expira.

4.3 Infraestrutura – Docker

A infraestrutura é orquestrada via Docker Compose com três serviços: db (PostgreSQL 16-alpine), backend (Python 3.12-slim + Gunicorn) e frontend (build multi-stage com Node 20-alpine e Ngnix-alpine).

O arranque do backend é sugerido pelo script entrypoin.sh, que aguarda a disponibilidade do PostgreSQL(healthcheck), executa as migrações, popula a base de dados com dados de demonstração (seed_data) e inicia o Gunicorn. O frontend Dockerfile utiliza uma build multi-stage: a primeira fase instala dependências e compila os assets React. A segunda fase copia o build para uma imagem Ngnix minimalista.

A configuração do Ngnix encaminha todos os pedidos com prefixo/api/ para o backend e serve a SPA React para todos os outros caminhos, garantindo o correto funcionamento do routing do React Router.

5 Resultados e Discussão

5.1 Funcionalidades Implementadas

A Tabela 5.1 apresenta as funcionalidades implementadas na plataforma ConstruManage com indicação do estado de conclusão.

Módulo	Funcionalidade	Estado
Autenticação	Login e registo com JWT	Concluído

Autenticação	Refresh automático de tokens	Concluído
Dashboard	Cards de indicadores em tempo rel	Concluído
Dashboard	Gráfico de barras receitas/despesas (6meses)	Concluído
Dashboard	Gráfico de pizza por categoria	Concluído
Obras	CRUD completo com controlo orçamental	Concluído
Obras	Barras de progresso e alertas de desvio	Concluído
Fluxo de Caixa	Registo de transações com categorização	Concluído
Fornecedores	Gestão de pagamentos com alertas de atraso	Concluído
Previsões	Projeções a 6 meses com análise de risco	Concluído
Infraestrutura	Orquestração Docker Compose	Concluído
Admin	Interface Django Admin	Concluído

Tabela 5.1 - Funcionalidades Implementadas

Todos os módulos planeados foram implementados e estão a funcionar. A plataforma pode ser iniciada com um único comando (Docker compose up – build -d) e é acessível imediatamente após o arranque dos serviços.

5.2 Testes

Os teste de funcionalidade foram realizados manualmente, verificando cada endpoint da API com o Postman e validando o comportamento da interface React nos principais browser(Chrome, Firefox, Edge). Foram ainda realizados testes de integração para validar o fluxo completo de autenticação e a correta propagação de dados entre frontend e backend.

Os dados de demonstração criados pelo seed_data incluem obras de construção com diferentes estados, transações financeiras distribuídas pelos últimos 6 meses, fornecedores com pagamentos pendentes e atrasados, previsões financeiras para semestre seguinte, permitindo validar todas as funcionalidades da plataforma com dados representativos.

6 Conclusão

6.1 Considerações Finais

O desenvolvimento da ConstrManage permitiu concretizar uma plataforma web completa e funcional para a gestão financeira de empresas de construção civil, cumprindo todos os requisitos definidos na fase de análise. A escolha do stack tecnológico(Django REST Framework, React, PostgreSQL, Docker) revelou-se adequada, proporcionando uma base sólida, modular e de fácil manutenção.

A plataforma responde eficazmente ao problema identificado – a falta de ferramentas adequadas para a monitorização financeira em tempo real no setor da construção civil, disponibilizando uma solução integrada, acessível e de fácil instalação. A arquitetura desacoplada, com backend e frontend independentes comunicando via API REST, garante flexibilidade para evolução futura de cada camada de forma independente.

Este projeto representou uma oportunidade valiosa de consolidar competências em desenvolvimento web full-stack, arquitetura de APIs REST, autenticação segura, contentorização com Docker e visualização de dados

6.2 Trabalho Futuro

Como prespetivas de trabalho futuro, destacam-se:

- Implementação de testes automatizados (unitários e de integração) com pytest-django e React Testing Library
- Adição de notificações por email para alertas de pagamento vencidos e desvios orçamentais
- Exportação de suporte multi-empresas com isolamento de dados por organização
- Deploy em produção com HTTPS, variáveis de ambiente seguras e backup automatizado da base de dados
- Desenvolvimento de uma aplicação móvel(React Native) para acesso em campo

Referências bibliográficas

[DJA 24] Django Software Fundation. Django Documentation 2026. Disponível em <https://docs.djangoproject.com/en/6.0/>

[DRF 24] Christie, T. et al. Django REST Framework Documentation 2026. Disponível em <https://www.django-rest-framework.org/>

[REA 24] Facebook Inc. Reac Documentation 2026. Disponível em <https://react.dev/>

[DOC 24] Docker Inc. Docker Documentation 2026. Disponível em <https://docs.docker.com/>

[JWT 24] M. Jones em JSON Web Token (JWT).RFC 7797.2016.IETF. Disponível em <https://datatracker.ietf.org/doc/html/rfc7797>

[POS 24] PostgreSQL Global Development Group. PostgreSQL Documentation 2026. Disponível em <https://www.postgresql.org/docs/>

Anexo A – API Endpoints

Este anexo lista todos os endpoints disponibilizados pela API REST da ConstruManage.

A.1 Autenticação

Método	Endpoint	Descrição	Auth
POST	api/auth/login/	Obter tokens JWT (access + refresh)	Não
POST	api/auth/refresh/	Renovar access token via refresh	Não
POST	api/auth/register/	Registar novo utilizador	Não

A.2 Recursos Protegidos

Módulo	Endpoint	Descrição
GET	api/dashboard/	Indicadores financeiros agregados
GET/POST	api/obras/	Listar/ criar obras
GET/PUT/DELETE	api/obras/{id}/	Detalhe/ editar/ eliminar obra
GET/POST	api/transacoes/	Listar/ criar transações
GET/PUT/DELETE	api/transacoes/{id}/	Detalhe/editar/eliminar transação
GET/POST	api/fornecedores/	Listar / criar fornecedores
GET/PUT/DELETE	api/fornecedores/{id}/	Detalhe / editar/ eliminar fornecedor
GET/POST	api/previsoes/	Listar / criar previsões
GET/PUT/DELETE	api/previsoes/{id}/	Detalhe / editar / eliminar previsões

Todos os endpoints da secção A.2 requerem o header de autenticação: Authorization: Bearer <access token>

